Enhancing Frontend-Backend Communication for Real-Time Data Analysis in Data Science Applications

Master Thesis by **Muhammadjon Yakhyoev**

Supervisor: dr hab.inż. Bartłomiej Zieliński

Faculty of Applied Sciences • Data Science • 2025

Introduction & Motivation

The Real-Time Expectation

Modern applications demand more than fast queries. Users expect **interactive dashboards**, instant feedback, and predictive indicators that refresh quickly enough for decision-making.

The Goal
Seamless UX combined with analytical depth

Integration Problem

Frontend must receive analytical results with low latency, while backend processes data reliably at scale.

ML Constraints

Introduces data leakage risks, versioning complexity, and computational overhead beyond standard CRUD.

Data Sync

Bulk ingestion and frequent updates can create mismatches between operational data and analytics.

Problem Statement

How to design frontend–backend communication and system architecture to deliver **real-time analytical and predictive capabilities** with acceptable latency, reliability, and methodological rigor?

Latency & Responsiveness

Predictions must feel "real-time" despite cross-service network overhead

Consistency & Synchronization

Analytical outputs must remain consistent with underlying operational data

Reliability & Security

Secure service communication and graceful failure handling

Scientific Rigor

Appropriate temporal validation and leakage prevention in production

Research Objectives

Architecture

Design a production-oriented architecture separating frontend, data layer, and ML computation.

Communication

Analyze how communication patterns affect latency and responsiveness for real-time analytics.

Machine Learning

Implement and evaluate supervised learning models with appropriate temporal validation.

Pipeline & Ingestion

Integrate bulk CSV ingestion as a realistic data pipeline component and evaluate its impact.

Reproducibility & Evaluation

Establish methodology for reproducible experiments, artifact management, and statistically meaningful interpretation.

The Rechly Platform

Case Study

A financial invoicing platform integrating **real-time predictive analytics**.

- Operational Workflows (Invoices, Clients, Products)
- Embedded Data Science Functionality
- Production-Ready Distributed Architecture

Late-Payment Prediction

Binary classification identifying invoices at risk of delayed payment.

Revenue Forecasting

Time-series prediction for 30- and 90-day cashflow planning.

Bulk Data Ingestion

CSV import pipeline for high-load data transfer and recalculation triggers.

System Architecture

Distributed ML-Enabled Web Application

Frontend

Next.js App

Dashboard • CRUD • Visualization

HOSTED ON

DigitalOcean

REST

Data Layer

Appwrite + Next.js API

Auth • Persistence • Business Logic

HOSTED ON

DigitalOcean

API

ML Microservice

Django REST

Training • Inference • Evaluation

HOSTED ON

Heroku

Cross-Cloud Communication: DigitalOcean ↔ Heroku | Secure Token-Based Auth

Machine Learning Models

Late-Payment Prediction

Binary Classification Task

MODEL 1

Logistic Regression

Interpretable, stable probabilities

MODEL 2

Gradient Boosting

Non-linear capacity, high performance

Objective: Estimate probability of payment delay > 7 days

Revenue Forecasting

Time-Series Prediction

PRIMARY

ML Regressor

Feature-based with lag features

BASELINE

Naive & Moving Average

Standard benchmarks for comparison

Horizon: 30-day and 90-day revenue predictions

Research Methodology

Temporal Splitting

Strict time-based splitting ensures no future information leaks into training.

- **Training Set**
  Earliest historical window
- **Validation Set**
  Subsequent window for tuning
- **Test Set**
  Latest window (unbiased)

Leakage-Safe

Features strictly enforce prediction-time availability to prevent look-ahead bias.

Key Principle

All aggregates at time t use only events $< t$

Invoice History | Client Behavior | Lag Features

Statistical Rigor

Validation of results through statistical significance testing and confidence intervals.

- **Paired Tests**
  Compare models on same periods
- **Bootstrap Methods**
  Robust confidence intervals
- **Baseline Comparison**
  vs Naive & Heuristic models

Experimental Evaluation

ML Model Performance

CLASSIFICATION RESULT

ML Models **Significantly Outperform** Heuristic Baselines

LATE-PAYMENT

ROC-AUC

LATE-PAYMENT

PR-AUC

REVENUE FORECAST

MAE / RMSE

REVENUE FORECAST

MAPE

System Latency

ARCHITECTURE RESULT

Acceptable Real-Time Inference in **Distributed Setup**

Prediction Request
Low-payload inference call
`Fast`

Forecast Retrieval
Analytical dashboard update
`Responsive`

Bulk CSV Ingestion
High-payload + ML recalc
`Scalable`

Conclusion & Future Work

Key Findings

- ML models significantly outperform baselines in late-payment prediction with rigorous validation.
- Distributed architecture (DigitalOcean + Heroku) delivers acceptable real-time latency.
- Leakage-safe feature engineering ensures credible financial predictions.

Contributions

- Practical design patterns for integrating predictive analytics into production web platforms.
- Demonstrated rigorous evaluation methodology under real system constraints.

Future Work

WebSocket Integration

Reduce latency with push-based updates.

Advanced Models

Explore deep learning for complex patterns.

Feature Store

Centralized feature management for consistency.

Enhanced Security

Zero-trust architecture for cross-cloud comm.

Thank You